Stepper Motor Shield — PID Position Control + S-Curve

PIC32MK0512MCJ064 firmware driving a NEMA17 stepper via TB6600 driver with AS5047U magnetic encoder feedback. Controlled over USB serial (115200 baud) or web dashboard.

Achieved accuracy: ±1–3 encoder counts = 0.022°–0.066° (limited by encoder noise floor).

Quick Start

```
# Flash firmware (via MDB)
rm -rf ~/.mplabcomm
/Applications/microchip/mplabx/v6.30/mplab_platform/
        bin/mdb.sh flash.mdb

# Connect serial
screen /dev/cu.usbmodemBUR2143200772 115200

# Start dashboard
cd dashboard && npm install && node server.js
# → http://localhost:3000
```

Power cycle the PKoB4 (unplug/replug USB) after flashing — VCOM gets stuck otherwise.

Serial Commands

All commands support both **legacy** and **Phase 1** formats. Legacy commands are preserved so the dashboard and existing scripts continue working.

Legacy Commands

Command	Action	Range
Position Control		
`T=<pos>`	Move to absolute position (encoder	±2M

Command	Action	Range
	counts, negative OK)	
`STOP`	Emergency stop — hold position, reset integrator	—
`HOME`	Set current position = 0 AND target = 0	—
`ZERO`	Set current position = 0 (leave target unchanged)	—
`CLEAR`	Clear fault, re-init motor driver	—
Open-Loop Motor Test		
`ON`	Enable motor + start spinning at last speed (default 100)	—
`OFF`	Disable motor, exit open-loop mode	—
`CW`	Set direction clockwise	—
`CCW`	Set direction counter-clockwise	—
`SPEED=<n>`	Set step rate for open-loop mode	10–100000
Motion Profile		
`MAXV=<n>`	Max velocity	≥1 steps/s
`ACCEL=<n>`	Max acceleration	≥100 steps/s²
`JERK=<n>`	Max jerk (S-curve smoothness)	≥1000 steps/s³
`PROFILE=S`	S-curve — jerk-limited smooth transitions	—
`PROFILE=T`	Trapezoidal — instant acceleration changes	—
PID Tuning		
`KP=<n>`	Proportional gain	0–99999
`KI=<n>`	Integral gain	0–99999
`KD=<n>`	Derivative gain	0–99999
`TOL=<n>`	At-target tolerance	0–1000 counts
`FT=<n>`	Fault threshold (checked only at rest)	1–100000 counts
Diagnostics		

Command	Action	Range
`GET`	One-shot config dump	—
Multi-Point Queue		
`Q=<pos>,<pos>,...`	Load queue (2–32 points) and start sequential execution	—
`DWELL=<ms>`	Set inter-point dwell delay (0–30000 ms)	—
`QSTOP`	Abort queue mid-execution	—

Phase 1 Commands (Colon-Separated)

Command	Action	Example
Move		
`m:<speed>:<pos>`	Move to position at given speed (sets MAXV temporarily)	`m:20000:80000`
Enable		
`en:1`	Enable closed-loop servo (hold current position)	`en:1`
`en:0`	Disable motor (release)	`en:0`
Zero		
`z`	Set current encoder position as zero	`z`
Coil Current (stored)		
`i:<mA>`	Set coil current limit (stored value, 100–5000 mA)	`i:800`
Microstep (stored)		
`us:<n>`	Set microstep setting (1/2/4/8/16/32)	`us:16`
PID Gains		
`pid:<kp>:<ki>:<kd>`	Set proportional, integral,	`pid:50:5:10`

Command	Action	Example
	derivative gains	
Telemetry		
`tlm:1:<ms>`	Enable status stream at N ms period	`tlm:1:50`
`tlm:0`	Disable status stream	`tlm:0`
Multi-Point Queue		
`q:<pos1>:<pos2>:...`	Load queue (2–32 points) and start sequential execution	`q:10000:20000:30000`
`dwell:<ms>`	Set inter-point dwell delay	`dwell:1000`
`qstop`	Abort queue mid-execution	`qstop`
Status Query		
`st?`	One-shot status: position, velocity, error, flags	`st?`

Response Format

All commands acknowledge:

`OK <command>=<value>\r\n`

or

`ERR UNKNOWN\r\n`

Automatic Status Stream (10 Hz default)

`P:<position>,E:<error>,V:<velocity>,T:<target>,F:<fault`

Set period with `tlm:1:<ms>` (10–10000 ms), disable with `tlm:0`.

st? Response (one-shot)

`p:<position>,v:<velocity>,e:<error>,f:<flags>\r\n`

Flags bitmask: - bit 0: fault - bit 1: moving - bit 2: at_target - bit 3: open_loop

GET Response

```
T=<target>,P=<position>,E=<error>,V=<velocity>,KP=<kp>,
T>,I=<mA>,US=<microstep>,QLEN=<n>,QIDX=<n>,DWELL=<ms>\r
```

Usage Examples

Basic Move

```
> en:1           Enable servo (holds current position)
OK en:1
> m:20000:50000  Move to position 50000 at 20000 steps/
s
OK m:20000:50000
> st?            Check status
p:50000,v:0,e:0,f:4
```

Snappy Move (profile feed-forward)

The motor runs at full commanded speed until the stopping distance (`v² / (2·accel_limit)`) from target, then decelerates hard. The PID acts as a trim, active only near the target.

```
> PROFILE=T      Trapezoidal profile (instant accel
changes)
> ACCEL=500000   Fastest ramp
> MAXV=50000     Top speed
OK PROFILE=T
OK ACCEL=500000
OK MAXV=50000
> m:50000:100000 Move at 50000 steps/s — reaches full
speed in ~100ms
OK m:50000:100000
```

PID Tuning

```
> pid:30:2:5     Set Kp=30, Ki=2, Kd=5
OK pid:30:2:5
> GET            Verify all params
T=10000,P=10000,E=0,V=0,KP=30,KI=2,KD=5,...
```

Auto-Tune

Relay-based Ziegler-Nichols PID tuning. The motor oscillates around a target, measures ultimate gain (Ku) and period (Tu), then applies:

```
Kp = 0.6 × Ku × 100    (capped at 10000)
Ki = 1.2 × Ku / Tu      (capped at 1000)
Kd = 0.075 × Ku × Tu    (capped at 10000)
```

Send `TUNE` to start. The motor will: 1. Move to current position + 3000 counts 2. Begin relay oscillation (±3000 steps/s relay velocity) 3. After 4+ full cycles, compute PID gains 4. Report: `OK TUNE:Kp=...,Ki=...,Kd=...,amp=...,Tu=...` 5. Return to original target

Caps on auto-tune output prevent excessive gains that cause oscillation — particularly Kd, which can destabilize the motor at high values.

Telemetry Stream

```
> tlm:1:200      Stream every 200ms
OK tlm:1:200
P:5000,E:-2,V:150,T:5000,F:0,M:0,A:1
P:5000,E:-1,V:0,T:5000,F:0,M:0,A:1
...
> tlm:0          Stop streaming
OK tlm:0
```

Multi-Point Queue

```
> DWELL=2000     Set 2 second pause between points
OK DWELL=2000
> Q=10000,30000,50000,20000  Load 4-waypoint queue
OK Q=4 points
> ...automatically moves: 10000 → 30000 → 50000 →
20000...
                Each arrival: 2s dwell → next move
> QSTOP          Abort mid-sequence
OK QSTOP
```

Queue is automatically aborted on any manual move (`T=`, `m:`), `STOP`, `OFF`, or `en:0`.

Accuracy

The system achieves **±1–3 encoder counts** following error at rest.

Domain	Value
1 encoder count	360° / 16384 = **0.022°**
Typical error	1–3 counts = **0.022°–0.066°**
AS5047U non-linearity	±0.05° = ~±2 counts
With 10mm leadscrew	1 count ≈ **0.6 µm**

The error is at the encoder's own noise floor. No further improvement possible without a higher-resolution encoder.

Non-Volatile Config

All tuning parameters (Kp, Ki, Kd, max_vel, accel_limit, jerk_limit, tolerance, fault_thr, profile, coil_current, microstep) are automatically saved to the last flash page when changed. Persist across power cycles.

- **Flash address**: last page of 512KB (0x1D07F000)
- **Save strategy**: 100ms debounce after last parameter change
- **Integrity**: magic word (0xBEADC0DE) + XOR checksum validated on load
- **Defaults**: used if flash is blank or corrupted

Motion Profiles

S-Curve (`PROFILE=S`)

Jerk-limited acceleration:

```
acceleration:  ──/‾‾‾‾‾‾\──  (ramps up/down at JERK rate)
velocity:      ──/‾‾‾‾‾‾\──  (rounded S-shaped transitions)
```

- Jerk limits how fast acceleration changes
- Lower JERK = smoother but slower moves
- Three phases: smooth accel → coast → smooth decel

Trapezoidal (`PROFILE=T`)

Instant acceleration changes:

```
acceleration:  ──┘‾‾‾‾‾‾└──  (square wave)
velocity:      ──/‾‾‾‾‾‾\──  (sharp V-shaped ramps)
```

- ACCEL limits the slope of velocity ramps
- Reaches target speed faster than S-curve at same ACCEL
- Higher mechanical stress at transitions

Profile Feed-Forward (v5.1.2+)

The velocity target is computed from a motion profile, not from PID error. The motor maintains full commanded speed until the stopping distance from target, then decelerates at the ACCEL rate. PID adds only a small trim for fine positioning.

```
raw_vel = profile_velocity(error) + pid_trim
profile_vel = max_vel (if far) or linear decel ramp (near target)
```

```
pid_trim    = (Kp·error + Ki·integral +
Kd·derivative) / 100
```

Anti-windup: integral resets during moves, only active near target. Auto-tune output capped at Kp≤10000, Ki≤1000, Kd≤10000 to prevent excessive gains.

Wiring

See PINOUT.md for complete pin mapping.

TB6600 Common-Anode Summary

TB6600	PIC32MK	Note
PUL+	RPB15 (OC1 PWM)	Step pulses, 50% duty
DIR+	RPB14 (GPIO)	LOW=CW, HIGH=CCW
ENA+	RPB13 (GPIO)	**LOW=enable**, HIGH=disable
PUL-/DIR-/ ENA-	GND	All three tied together

TB6600 DIP Switches

- S1/S2/S3: per motor current rating
- S4/S5/S6: S4 ON, S5 ON, S6 ON = 1/32 microstep (6400 steps/ rev)

Dashboard

Web-based UI on port 3000:

```
dashboard/
├── server.js          # Express + WebSocket + serial
bridge
├── package.json
└── public/
    ├── index.html     # UI layout
    ├── app.js         # Client logic
    └── styles.css     # Dark theme
```

Features

- Position display: target / actual with bar indicator
- Status: error, velocity, fault, moving, at-target
- Motor Test card: ON/OFF, CW/CCW, speed slider (up to 50000 steps/s)

- PID sliders: Kp (0–50000), Ki (0–10000), Kd (0–50000)
- Auto-tune button: relay-based Ziegler-Nichols tuning, results shown with slider label sync
- TLM toggle: enable/disable telemetry stream
- Motion Profile: S-curve / Trapezoidal toggle
- ACCEL and JERK number inputs with Set buttons
- MAXV, Tolerance, Fault Threshold inputs
- GET response parsing: position, error, velocity, TOL, FT fields

Architecture

```
Firmware (PIC32MK) ↔ Serial (115200) ↔ server.js ↔
WebSocket ↔ browser
```

Firmware Architecture

Component	Peripheral	Rate	Purpose
Position loop	Timer3 ISR	1 kHz	Profile → PID trim → smoother → velocity
Step generation	OC1 + Timer2 PWM	Variable	Hardware step pulses, 50% duty
Encoder	QEI1	Continuous	AS5047U ABI, 16384 counts/rev
UART	UART1	115200 baud	Command + telemetry
NVM flash	Self-write	On change	Config storage (last flash page)
Status stream	Main loop	Configurable	P:E:V:T:F:M:A:Q:QL telemetry

Algorithm

```
error = target_pos − encoder_read()

// Profile feed-forward velocity
stop_dist = max_vel² / (2 × accel_limit)     //
stopping distance
if abs(error) > stop_dist:
    profile_vel = max_vel × sign(error)       // cruise
else:
    profile_vel = max_vel × error / stop_dist // decel
ramp

// PID trim
pid_trim = (Kp×error + Ki×integral + Kd×derivative) /
100
```

```
// Combined, fed into accel/jerk smoother
raw_vel = profile_vel + pid_trim

// Accel/jerk smoother → sm_vel → motor steps
```

Resolution

Aspect	Value
Motor steps/rev	200 (NEMA17, 1.8°/step)
Microstep setting	**1/32** (S4 ON, S5 ON, S6 ON)
Steps per revolution	**6400**
Encoder PPR	4096 (AS5047U)
Encoder counts/rev (4× QEI)	16384
Position resolution	360° / 16384 = **0.022°** per count

Voltage-Dependent Limits

At **1/32 microstep** (6400 steps/rev). Firmware accepts any positive value; the practical limit is set by the motor driver and supply voltage. Empirically determined guidelines (no load, idle condition):

Supply Voltage	MAXV (steps/s)	MAXV (RPM)	ACCEL (steps/s²)	Notes
31V	280,000	2,625	2,700,000	Best reliable: 1725 mm/s, Kp=2121 Ki=46
24V	250,000	2,344	3,200,000	At ≥2.5A coil current, smooth with heating
12V	50,000	469	5,000,000	Reduced max velocity

31V characterization (from `docs/TESTS.md`): - MAXV 300k with 3.2M ACCEL reaches 1902 mm/s (Kp 50–1000, Ki=5, Kd=0) - Kp >~2000 with Ki > 0 causes oscillation at 31V; best results use moderate Kp - Trade-off: higher MAXV requires lower ACCEL and vice versa - Kd was 0 in all 31V tests — derivative not needed with proper Kp/Ki balance

RPM formula: `RPM = (steps/s × 60) ÷ steps_per_rev` where `steps_per_rev = 6400` at 1/32.

Releases

Tag	Date	Description
v6.2.0	2026-06-20	**Auto-tune caps restored, 31V test data.** Kp≤10000/Ki≤1000/Kd≤10000 caps restored on auto-tune output (prevents oscillation from excessive gains). 31V characterization: 280k MAXV / 2.7M ACCEL / 1725 mm/s reliable belt speed.
v6.1.0	2026-06-17	**SPLL clock (48 MHz), 115200 baud, no command limits.** FRC→SPLL migration. All clock timing fixed. MAXV/ACCEL/JERK upper limits removed. Dashboard TLM toggle.
v6.0.0	2026-06-15	**Phase 1 complete.** `docs/BUGS_AND_LESSONS.md` — full 24+ bug knowledge base. Dashboard activeElement input protection.
v5.2.0	2026-06-12	Multi-point queue (Q=/q:), DWELL, QSTOP, dashboard queue card
v5.1.4	2026-06-12	Dashboard UI ranges match firmware; ACCEL/JERK number inputs; accuracy ±1–3 counts
v5.1.3	2026-06-12	Snappier defaults: MAXV=5000, ACCEL=50000, JERK=500000; ranges increased 5–10×
v5.1.2	2026-06-12	Profile feed-forward velocity; full speed until stopping distance; PID trim only near target
v5.1.1	2026-06-12	NVM flash config: all settings persist across power cycles
v5.1.0	2026-06-12	Phase 1 commands (colon-separated), Kd, configurable telemetry
v5.0.1	2026-06-11	Trapezoidal profile, dashboard toggle, ACCEL/JERK commands
v5.0.0	2026-06-11	PID + S-curve + open-loop test + 6 bug fixes
v4.0.0	2026-06-10	UART serial fixes
v3.1.0	2026-06-09	Open-loop speed control

Tag	Date	Description
v3.0.0	2026-06-09	Closed-loop position control
v2.0.0	2026-06-09	Stepper control + encoder
v1.0.0	2026-06-08	Encoder-only firmware

TB6600 + NEMA17 Motor Test — Journey Log

Overview

Goal: Drive a NEMA17 stepper motor using a TB6600 driver, controlled first by an Arduino Uno for validation, then by a PIC32MK0512MCJ064 on a Curiosity Pro board with closed-loop encoder feedback.

Phase 1: Arduino Prototyping

1.1 Initial Wiring (Broken)

First attempt with the TB6600's common-anode wiring (all − signals tied to GND, + signals driven by MCU):

TB6600	Arduino	Note
PUL+	D3	Step pulse signal
DIR+	D4	Direction: LOW=CW, HIGH=CCW
ENA+	**D5**	❌ Initially set HIGH to "enable" — wrong
PUL-/DIR-/ ENA-	GND	All three commons tied together
V+	9-42V DC supply	
A+/A-	NEMA17 coil A	
B+/B-	NEMA17 coil B	

Symptom: Motor vibrated/hummed but did not spin. Step LED on TB6600 was solid (not blinking).

1.2 ENA Polarity Fix

Root cause: In common-anode wiring, the TB6600 optocoupler works in reverse:

- **ENA+ LOW** → optocoupler OFF → **driver enabled** ✅
- **ENA+ HIGH** → optocoupler ON → **driver disabled** ❌

This is the opposite of common-cathode wiring and opposite of the typical TB6600 documentation.

Fix: Set `ENA_PIN = LOW` to enable. Motor immediately spun.

1.3 Arduino Test Code

File: `tests/arduino/motor_test.ino`

Final test sequence (verified working at 10 kHz step rate): 1. 16000 steps CW (~2.5 revs at 1/32 microstep = 6400 steps/rev), pause 2 s 2. 16000 steps CCW, pause 2 s 3. Loop forever

Step pulse: 50 µs HIGH, 50 µs LOW → 100 µs period → **10 kHz** step rate.

1.4 PlatformIO Build

File: `tests/arduino/platformio_test/`

PlatformIO project for Arduino Uno clone (ATmega16U2 USB, 57600 baud upload).

Phase 2: PIC32MK Integration

2.1 Pin Mapping

After Arduino validation, the same control scheme was ported to PIC32MK using hardware-generated step pulses (no CPU-wasting `digitalWrite` loops):

Signal	PIC32MK Pin	Function	TB6600
STEP	**RPB15** (pin 44)	OC1 — Output Compare PWM	PUL+
DIR	**LATB14** (pin 43)	GPIO	DIR+
ENA	**LATB13** (pin 42)	GPIO (LOW = enable)	ENA+
GND	GND		

Signal	PIC32MK Pin	Function	TB6600
			PUL-/DIR-/ ENA-

2.2 Hardware Step Generation

Instead of bit-banging step pulses (Arduino approach), the PIC32MK uses **OC1** in PWM mode clocked by **Timer2**:

- Timer2 prescaler = 1:1, clocked at PB1CLK = 4 MHz
- PR2 = `(PB_CLOCK / STEP_RATE) - 1`
- OC1R = OC1RS = PR2 >> 1 (50% duty cycle)
- OC1 output mapped to RPB15 via `RPB15R = OC1_FN` (PPS remapping)

The CPU is free to poll the encoder, drive UART, etc. while the hardware generates precise step pulses.

2.3 Encoder Feedback

AS5047U magnetic encoder in ABI mode connected to QEI1: - A → RPG6 (QEA1), B → RPG7 (QEB1) - 4096 PPR, 4× decoding → 16384 counts/rev - RPM calculated from position delta over 1 s sample window

2.4 Current Firmware Behavior

`newxc32_newfile.c` main loop: 1. Initialize UART (19200 baud), QEI encoder, and motor driver 2. Run **CW for 5 seconds**, report RPM via UART every second 3. Run **CCW for 5 seconds**, report RPM via UART every second 4. LED (RA10) toggles each second as heartbeat

---

Phase 3: Closed-Loop Position Control

Architecture

The firmware now runs a **1 kHz position control loop** (Timer3 ISR) that:

1. Reads actual position from QEI encoder (AS5047U, 16384 counts/rev)
2. Computes following error: `error = target_position - actual_position`
3. Checks fault threshold: if `|error| > FT`, triggers emergency stop
4. Runs PI controller: `velocity = Kp × error + Ki × ∫error`
5. Clamps velocity to `MAXV` (coil current limit enforcement)
6. Sets direction and OC1 step rate via hardware PWM

7. When within tolerance, stops OC1 and disables driver (position hold)

UART Protocol

The firmware communicates with the dashboard at 19200 baud:

Status telemetry (10 Hz):
`P:<pos>,E:<error>,V:<vel>,T:<target>,F:<fault>,H:<homed>,M:<moving>`

Commands:
- `T=<pos>` — Set target position (triggers point-to-point move) - `HOME` — Set current position as zero reference - `STOP` — Emergency stop (motor off, target = current position) - `CLEAR` — Clear fault and reinitialize motor pins - `KP=<val>` — Set proportional gain (default 50) - `KI=<val>` — Set integral gain (default 5) - `MAXV=<val>` — Set max velocity in steps/s (default 2000) - `TOL=<val>` — Set position tolerance in counts (default 5) - `FT=<val>` — Set fault threshold in counts (default 500)

Dashboard

Web-based control running on Node.js/Express with WebSocket: - Real-time actual/target position bars - Following error gauge (positive/negative/zero) - Status LEDs: homed, moving, at-target, fault - Velocity readout - Target position input + GO button - STOP, HOME, Clear Fault buttons - Live tuning sliders for Kp, Ki, MaxV, Tolerance, Fault Threshold

---

Key Lessons

Issue	Lesson
ENA polarity	Common-anode wiring: LOW = enable, HIGH = disable. Check your optocoupler wiring!
PPS unlock	PIC32MK has three lock bits: PGLOCK, PMDLOCK, IOLOCK. All must be cleared.
PB1 clock divider	Default PB1DIV = SYSCLK/2. UART baud calculations must use actual PB1CLK.
CFGCON address	`0xBF800000` (not `0xBF80F610`) for PIC32MK family. Wrong address causes crash.
Watchdog	Config bits alone may not disable WDT reliably — use section-based config words.

Issue	Lesson
OC1 vs MC PWM	OC1 is simple (2 register writes + PPS mapping). MC PWM module is complex (dead-time, fault handling) and overkill for step pulses.

Motor Test Results — Test Bench Data 1

Constant PID: Kp=10, Ki=5, Kd=0, 24V supply. Varying MAXV, Accel, and coil current to characterize high-speed limits.

MAXV (steps/s)	Belt Speed (m/s)	RPM	Accel (steps/s²)	Kp	Ki	Kd	V	Coil Current (A)	Idle A	Result
250,000	1.56	2,344	3,200,000	10	5	0	24V	3.5–4	0.676	Smooth with heating
250,000	1.56	2,344	3,200,000	10	5	0	24V	3–3.2	0.662	Smooth with heating
250,000	1.56	2,344	3,200,000	10	5	0	24V	2.8–2.9	0.642	Smooth with heating
250,000	1.56	2,344	3,200,000	10	5	0	24V	2.5–2.7	0.550	Smooth with heating
230,000	1.44	2,156	3,300,000	10	5	0	24V	2–2.2	0.292	Smooth with heating
200,000	1.25	1,875	1,900,000	10	5	0	24V	1.5–1.7	0.223	Minute Shuttering
200,000	1.25	1,875	190,000	10	5	0	24V	1–1.2	0.112	**Motor Stuck**

Conclusion: At 250k steps/s (3.2M steps/s² accel), minimum reliable coil current is 2.5–2.7A. At 230k/3.3M, 2–2.2A is sufficient. Below 2A with high speed, shuttering or stalling occurs.

Phase 4: v6.1.0 — SPLL Clock, 115200 Baud, No Limits

Clock Migration (FRC → SPLL)

The firmware was migrated from FRC (8 MHz) to SPLL (48 MHz) to enable reliable 115200 baud serial communication:

Parameter	v6.0.0 (FRC)	v6.1.0 (SPLL)
System clock	8 MHz (FRC)	48 MHz (PLLIDIV=2, PLLMULT=24, PLLODIV=2)
PB clock	4 MHz	48 MHz
Core timer	4 MHz	24 MHz
Timer2 (step)	500 kHz (TCKPS=3)	6 MHz (TCKPS=3, same prescaler)
UART baud	19200	115200 (U1BRG=25, 0.16% error)

All Clock-Dependent Formulas Updated

Eight formulas were adjusted to match the new clock speeds:

1. **`msleep()`**: `ms * 4000u` → `MS(ms)` (core timer 4 MHz → 24 MHz)
2. **`motor_set_speed()`**: `500000 / sps` → `(PB_CLOCK / 8) / sps` (Timer2 500 kHz → 6 MHz)
3. **Auto-tune Tu**: `/4000.0f` → `/ ((float)TICKS_PER_MS * 1000.0f)` (core timer)
4. **Config save**: `PB_CLOCK / 10` → `MS(100)` (3× faster core timer)
5. **`tlm_ticks`**: `(uint32_t)` cast added for overflow safety
6. **`CONFIG_DATA_WORDS`**: 11 → 12 (struct padding fix for checksum)
7. **Timer3 PR3**: auto-scaled by `PB_CLOCK / CONTROL_FREQ` (already formula-based)

Removed Command Limits

All hard-coded upper bounds removed from firmware:

Parameter	Old limit	New limit
MAXV	≤ 100,000 steps/s	Any positive value
ACCEL	≤ 5,000,000 steps/s²	≥ 100
JERK	≤ 10,000,000 steps/s³	≥ 1000
m: speed	≤ 100,000 steps/s	Any positive value

Belt Speed Example (G2 Pulley, 20 Teeth × 2 mm)

Given `MAXV=190000`, `ACCEL=1800000`, `JERK=500000`:

Calculation	Value
Pulley circumference	20 × 2 mm = **40 mm/rev**
Steps per rev	200 steps × 32 microsteps = **6400 steps/rev**
Steps per mm	6400 ÷ 40 = **160 steps/mm**
Belt speed at MAXV	190,000 ÷ 160 = **1,187.5 mm/s = 1.19 m/s**
Accel in mm/s²	1,800,000 ÷ 160 = **11,250 mm/s² ≈ 1.15 g**
Motor RPM	190,000 ÷ 6400 × 60 = **1,781 RPM**

Dashboard Updates

- Default baud: 115200 (server.js, app.js, index.html)
- Telemetry toggle button (TLM ON/OFF)
- GET response parsing: position, error, velocity, TOL, FT fields

Flash Script

`flash.mdb` — MDB programming script for command-line flashing:

```
rm -rf ~/.mplabcomm
/Applications/microchip/mplabx/v6.30/mplab_platform/
        bin/mdb.sh flash.mdb
# Must power-cycle PKoB4 after flashing (VCOM gets
        stuck)
```

Motor Test Results — 31V Test Data

Constant PID varies. 31V supply. Systematic sweep of ACCEL, MAXV, and PID gains to characterize high-speed performance at 31V.

Voltage	Accel (steps/s²)	MAXV (steps/s)	Kp	Ki	Kd	Measured Velocity	Target (counts)	Result
31V	3,200,000	300,000	50	5	0	1670 mm/s	25,000	Smooth with heating
31V	3,200,000	300,000	50	5	0	1780 mm/s	25,000	Smooth with heating
31V	3,200,000	300,000	1000	5	0		25,000	

Voltage	Accel (steps/s²)	MAXV (steps/s)	Kp	Ki	Kd	Measured Velocity	Target (counts)	Result
						1902 mm/s		Smooth with heating
31V	1,400,000	300,000	1092	0	0	1445 mm/s	25,000	Smooth with heating
31V	1,700,000	300,000	1092	46	0	1610 mm/s	25,000	Not Smooth
31V	1,700,000	300,000	3000	46	0	1543 mm/s	25,000	Smooth with oscillation
31V	1,800,000	300,000	3000	46	0	—	25,000	**Stuck**
31V	1,700,000	310,000	3000	46	0	1470 mm/s	25,000	**Stuck**
31V	2,000,000	240,000	1050	46	0	1440 mm/s	25,000	Smooth with heating
31V	2,700,000	270,000	1934	46	0	1669 mm/s	25,000	Smooth with heating
31V	2,700,000	280,000	2121	46	0	1725 mm/s	25,000	Smooth with heating

Key Findings at 31V: - **300k MAXV / 3.2M ACCEL** works with moderate Kp (50–1000) and Ki=5, Kd=0 — best measured velocity 1902 mm/s - **High Kp (3000) + Ki (46)** causes oscillation or stalling — Kp >~2000 at 31V is unstable with nonzero Ki - **MAXV > 300k requires lower ACCEL** trade-off: 280k MAXV + 2.7M ACCEL works, 310k MAXV + 1.7M ACCEL does not - **Best reliable result:** 270k MAXV + 2.7M ACCEL, Kp=2121, Ki=46 — 1725 mm/s smooth - **Kd=0** in all tests — derivative gain not needed at 31V with proper Kp/Ki balance

---

File Reference

File	Description
`newxc32_newfile.c`	PIC32MK firmware: stepper + encoder + UART (v6.1.0)
`flash.mdb`	MDB command-line flash script
`blink_test.c`	Minimal GPIO blink test
`docs/PINOUT.md`	Wiring reference
	Arduino validation test

File	Description
`tests/arduino/motor_test.ino`	
`tests/arduino/platformio_test/`	PlatformIO project for Arduino test

Pinout Reference

PIC32MK0512MCJ064 → TB6600 Stepper Driver

Common-Anode Wiring

All TB6600 **−** pins are tied together to **GND**. All **+** pins are driven by PIC32MK GPIO.

Critical: ENA+ LOW = enabled, HIGH = disabled. Do NOT tie ENA- to ENA+ or bypass the optocoupler — the TB6600's internal circuit needs PUL-/DIR-/ENA- as the return path. Connecting ENA- alone without PUL-/DIR- also tied to GND will not work.

TB6600	PIC32MK Pin	Pin # (64-TQFP)	Notes
PUL+	**RPB15** (OC1)	44	50% duty PWM, rate = `STEP_RATE` Hz
DIR+	**RPB14**	43	LOW = CW, HIGH = CCW
ENA+	**RPB13**	42	**LOW = enabled**, HIGH = disabled
PUL-	GND		
DIR-	GND		
ENA-	GND		

Physical Pin Identifiers

Pin 44 on the 64-TQFP package is labeled as **PB15 / RB15 / PWML1** (different functions on same pad). The firmware routes OC1 output to RPB15 via PPS — no connection to the dedicated Motor Control PWM module is needed.

Other Connections

Function	PIC32MK Pin	Pin #	Notes
UART TX	**RPE0**	1	Debug output, 19200 baud
UART RX	**RPG8**	54	(via PPS input #1)
QEI A	**RPG6**	52	AS5047U encoder channel A
QEI B	**RPG7**	53	AS5047U encoder channel B
LED	**RA10**	28	Heartbeat/status indicator

TB6600 DIP Switch Settings

Set DIP switches to match NEMA17 motor rated current and desired microstep resolution:

Switch	Setting	Effect
S1/S2/S3	Per motor current	Match NEMA17 rating plate
S4/S5/S6	Per microstep	1/32 = S4 ON, S5 ON, S6 ON (6400 steps/rev)

NEMA17 → TB6600

TB6600	NEMA17
A+	Coil A+ (typically red)
A-	Coil A- (typically blue)
B+	Coil B+ (typically green)
B-	Coil B- (typically black)
V+	9-42V DC supply positive
GND	DC supply negative